Ex. No.	6	Name	Ajmal Rahman A
Date	18-10-2026	Reg. No.	192324094
Sub. Name	Deep Learning	Sub. Code	CSA4702
Semester	MS2 September 2026		

EXERCISE NAME

GRADIENT DESCENT WITH MOMENTUM FOR AN OSCILLATING LOSS

SCENARIO

A neural network experiences oscillations while optimising its loss function, because its input features are measured on very different scales. Implement gradient descent with momentum and compare its convergence with that of standard gradient descent, using the same starting point, the same learning rate and the same number of epochs.

ALGORITHM

Step 1 : Import the packages used in this exercise — NumPy for the matrix algebra and Matplotlib for the loss curves and the contour plot of the loss surface.

Step 2 : Build a training problem that deliberately oscillates. Generate 200 samples with two input features, of which the first is well scaled with a standard deviation near 0.93 and the second is badly scaled with a standard deviation near 0.07, and obtain the target from the true weight vector $w^* = (2.5, -4.0)$. The mismatch of the two scales is what turns the loss surface into a long narrow valley.

Step 3 : Define the loss as the mean squared error, $L(w) = (1/m)\ \Sigma\ (X \cdot w - y)^2$, and its gradient as $\nabla L(w) = (2/m)\ X^T\ (X \cdot w - y)$.

Step 4 : Compute the Hessian of the loss surface, $H = (2/m)\ X^T X$, together with its eigenvalues and its condition number $\kappa = \lambda_{max} / \lambda_{min}$. A large condition number means the surface is far steeper along one direction than the other, and gradient descent stays stable only while the learning rate is below $2 / \lambda_{max}$.

Step 5 : Fix the starting point $w_0 = (-6, 8)$, the learning rate $\alpha = 1.05$, which is just below the stability limit, and 300 epochs. Use the same three values for both methods so that the comparison is fair.

Step 6 : Run standard gradient descent with the update $w := w - \alpha\ \nabla L(w)$, recording the loss and the weight vector after every epoch.

Step 7 : Run gradient descent with momentum with the two updates $v := \beta v + \nabla L(w)$ and $w := w - \alpha v$, taking $\beta = 0.70$ and $v = 0$ at the start. The velocity v carries a fraction β of the previous step into the present one, so the component of the gradient that keeps reversing its sign cancels itself, while the component that keeps its direction accumulates and grows.

Step 8 : Measure the behaviour of both methods — the number of times the step along the steep weight w_1 reverses its direction, the number of epochs in which the loss rose instead of falling, the number of epochs needed to bring the loss below 10^{-3}, and the final distance $\|w - w^*\|$ from the optimum.

Step 9 : Repeat the momentum run with β = 0.00, 0.50, 0.70, 0.90, 0.95 and 0.99 to study how the coefficient controls the damping, noting that β = 0 reduces the method to standard gradient descent and that a value too close to 1 makes the steps overshoot.

Step 10 : Plot the loss of both methods against the epoch on a logarithmic scale.

Step 11 : Plot the contour of the loss surface with the first 60 epochs of both paths drawn on it, so that the zig-zag of standard gradient descent and the damped path of momentum can be compared directly. Stop.

SOURCE CODE

Module 1 : Importing the Required Packages

```
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams["font.family"]          = "serif"
plt.rcParams["font.serif"]           = ["Cambria", "Caladea", "DejaVu Serif"]
plt.rcParams["axes.spines.top"]      = False
plt.rcParams["axes.spines.right"]    = False
plt.rcParams["legend.frameon"]       = False
```

Module 2 : Building an Ill-Conditioned Training Problem

```
np.random.seed(42)
N = 200                                              # number of training samples

x1 = np.random.normal(0, 1.0, N)                      # well-scaled feature
x2 = np.random.normal(0, 0.07, N)                     # badly scaled feature
X  = np.column_stack([x1, x2])                    # input matrix , shape (200, 2)

W_TRUE = np.array([2.5, -4.0])                        # weights to be recovered
y = X @ W_TRUE                                    # target vector , shape (200,)

print("Input matrix X :", X.shape, "  Target y :", y.shape)
print("Standard deviation of the two features :", np.round(X.std(axis=0), 4))
print("True weights :", W_TRUE)
```

Module 3 : Loss Function, Gradient and the Conditioning of the Surface

```
def loss(w):
    """Mean squared error of the single neuron for the weight vector w."""
    r = X @ w - y
    return float(np.mean(r ** 2))

def grad(w):
    """Gradient of the mean squared error with respect to w."""
    r = X @ w - y
    return (2.0 / N) * (X.T @ r)

H      = (2.0 / N) * (X.T @ X)                     # Hessian of the loss surface
eig    = np.linalg.eigvalsh(H)
kappa  = eig.max() / eig.min()                       # condition number

print("Hessian H :\n", np.round(H, 4))
print("Eigenvalues of H :", np.round(eig, 6))
print("Condition number : %.2f  (large value -> elongated valley)" % kappa)
print("Stable learning-rate limit 2/L : %.4f" % (2.0 / eig.max()))
```

Module 4 : Standard Gradient Descent

```
W0      = np.array([-6.0, 8.0])                  # same starting point for both
```

```
LR      = 1.05                              # a rate that makes plain GD oscillate
EPOCHS  = 300

def gradient_descent(lr, epochs):
    w, hist, path = W0.copy(), [], [W0.copy()]
    for _ in range(epochs):
        g  = grad(w)
        w -= lr * g                                     # w := w - alpha * grad
        hist.append(loss(w));  path.append(w.copy())
    return w, np.array(hist), np.array(path)

w_gd, loss_gd, path_gd = gradient_descent(LR, EPOCHS)
print("Standard GD  -> final loss %.6e  final weights %s"
      % (loss_gd[-1], np.round(w_gd, 4)))
```

Module 5 : Gradient Descent with Momentum

```
def momentum_descent(lr, epochs, beta):
    w, v, hist, path = W0.copy(), np.zeros(2), [], [W0.copy()]
    for _ in range(epochs):
        g = grad(w)
        v = beta * v + g                    # velocity accumulates the gradients
        w -= lr * v                                     # w := w - alpha * v
        hist.append(loss(w));  path.append(w.copy())
    return w, np.array(hist), np.array(path)

BETA = 0.70
w_mo, loss_mo, path_mo = momentum_descent(LR, EPOCHS, BETA)
print("Momentum GD  -> final loss %.6e  final weights %s"
      % (loss_mo[-1], np.round(w_mo, 4)))
```

Module 6 : Measuring the Oscillation and the Speed of Convergence

```
def oscillations(path):
    """Times the step along the steep weight w1 reverses direction."""
    s = np.diff(path, axis=0)[:, 0]
    return int(np.sum(np.sign(s[1:]) != np.sign(s[:-1])))

def epochs_to(hist, tol):
    idx = np.where(hist < tol)[0]
    return int(idx[0] + 1) if idx.size else -1

TOL = 1e-3
for name, hist, path, w in [("Standard GD", loss_gd, path_gd, w_gd),
                            ("Momentum GD", loss_mo, path_mo, w_mo)]:
    print("%-12s | osc %4d | epochs to loss < %.0e : %4d | final %.3e"
          % (name, oscillations(path), TOL, epochs_to(hist, TOL), hist[-1]))
```

Module 7 : Effect of the Momentum Coefficient

```
betas, table = [0.0, 0.50, 0.70, 0.90, 0.95, 0.99], []
for b in betas:
    w_b, h_b, p_b = momentum_descent(LR, EPOCHS, b)
    table.append((b, h_b[-1], epochs_to(h_b, TOL), oscillations(p_b),
                  float(np.linalg.norm(w_b - W_TRUE))))
    print("beta %.2f | loss %.4e | epochs %4d | osc %4d | |w-w*| %.4f"
          % table[-1])
```

Module 8 : Visualisation of the Convergence

```
ep = np.arange(1, EPOCHS + 1)
fig, ax = plt.subplots(figsize=(6.6, 3.4))
ax.plot(ep, loss_gd, color="#C0702A", lw=1.5, label="Standard gradient descent")
ax.plot(ep, loss_mo, color="#2F6DB5", lw=1.5,
        label="Gradient descent with momentum ($\\beta$ = %.2f)" % BETA)
ax.set_yscale("log")
```

```
ax.set_xlabel("Epoch");  ax.set_ylabel("Mean squared error (log scale)")
ax.set_xlim(0, EPOCHS)
ax.grid(alpha=0.55);  ax.set_axisbelow(True)
ax.legend(loc="upper right", fontsize=8)
fig.tight_layout()
fig.savefig("plot1_loss_curves.png", dpi=300, bbox_inches="tight")

g1 = np.linspace(-8, 11, 220)
g2 = np.linspace(-9, 10, 220)
G1, G2 = np.meshgrid(g1, g2)
Z = np.array([[loss(np.array([a, b])) for a in g1] for b in g2])

fig, ax = plt.subplots(figsize=(6.0, 4.4))
cs = ax.contour(G1, G2, Z, levels=np.geomspace(Z.min() + 1e-3, Z.max(), 16),
                colors="#B8B8B8", linewidths=0.6)
ax.plot(path_gd[:60, 0], path_gd[:60, 1], "-o", color="#C0702A", lw=1.0, ms=2.4,
        label="Standard gradient descent")
ax.plot(path_mo[:60, 0], path_mo[:60, 1], "-o", color="#2F6DB5", lw=1.0, ms=2.4,
        label="Gradient descent with momentum")
ax.scatter(*W0, s=42, c="#4E8F59", marker="s", zorder=4, label="Starting point")
ax.scatter(*W_TRUE, s=60, c="#1A1A1A", marker="*", zorder=4, label="Optimum")
ax.set_xlabel("Weight $w_1$");  ax.set_ylabel("Weight $w_2$")
ax.set_title("First 60 epochs on the loss surface", fontsize=10)
ax.set_xlim(-8, 11);  ax.set_ylim(-9, 10)
ax.legend(loc="lower left", fontsize=7.5)
fig.tight_layout()
fig.savefig("plot2_trajectory.png", dpi=300, bbox_inches="tight")
```

OUTPUT

Table 1 - Training Data Sample (First 10 Samples)

S.No	Sample	Feature x_1	Feature x_2	Target y
1	S001	+0.4967	+0.0250	+1.1416
2	S002	-0.1383	+0.0393	-0.5027
3	S003	+0.6477	+0.0758	+1.3160
4	S004	+1.5230	+0.0738	+3.5125
5	S005	-0.2342	-0.0964	-0.1996
6	S006	-0.2341	-0.0656	-0.3228
7	S007	+1.5792	+0.0361	+3.8038
8	S008	+0.7674	+0.0360	+1.7747
9	S009	-0.4695	+0.0361	-1.3179
10	S010	+0.5426	+0.2697	+0.2776

Table 2 - Conditioning of the Loss Surface (the Cause of the Oscillation)

Parameter	Value	Remark
Standard deviation of x_1	0.9287	Well scaled
Standard deviation of x_2	0.0689	Badly scaled
Hessian $H = (2/m)\ X^TX$	[[1.7282, 0.0117], [0.0117, 0.0096]]	Curvature of the loss
Larger eigenvalue λ_{max}	1.728273	Steep direction
Smaller eigenvalue λ_{min}	0.009492	Flat direction
Condition number κ	182.08	Elongated narrow valley
Stability limit $2 / \lambda_{max}$	1.1572	Learning rate used is 1.05
Starting point w_0	[-6.0, 8.0]	Same for both methods
Optimum w^*	[2.5, -4.0]	True weights

Table 3 - Loss per Epoch : Standard Gradient Descent against Momentum

S.No	Epoch	Standard GD Loss	Momentum GD Loss	Reduction Factor
1	1	41.320749	41.320749	1.00 ×
2	5	8.511477	15.153524	0.56 ×
3	10	1.580583	0.469641	3.37 ×
4	20	0.479059	0.216437	2.21 ×
5	40	0.309627	0.043835	7.06 ×
6	60	0.207408	0.009917	20.91 ×
7	80	0.138938	0.002247	61.83 ×
8	100	0.093071	0.000509	182.81 ×
9	150	0.034182	0.000012	2747.70 ×
10	200	0.012554	0.000000	41298.70 ×
11	250	0.004611	0.000000	620731.20 ×
12	300	0.001693	0.000000	9329766.18 ×

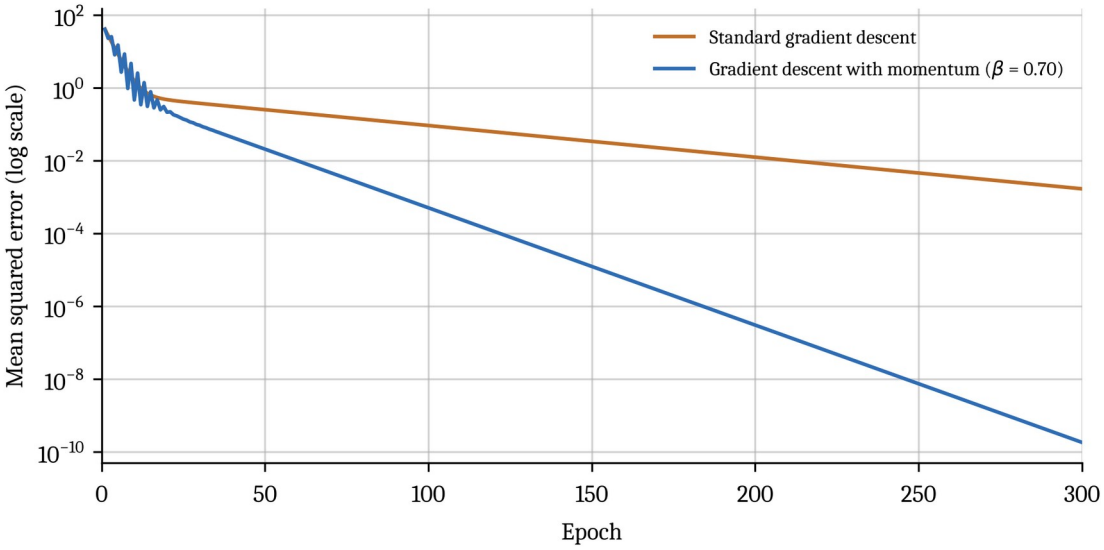
Table 4 - Convergence Summary of the Two Methods

Parameter	Standard Gradient Descent	Gradient Descent with Momentum
Learning rate α	1.05	1.05
Momentum coefficient β	— (not used)	0.70
Epochs executed	300	300
Loss at epoch 1	41.320749	41.320749
Loss at epoch 50	0.253413	0.020836
Loss at epoch 300	1.693e-03	1.815e-10
Epochs to reach loss $< 10^{-3}$	Not reached	91
Direction reversals (oscillations)	50	30
Epochs in which the loss rose	0	10
Final weight w_1	2.4959	2.5000
Final weight w_2	-3.4027	-3.9998
Distance from the optimum $\|w - w^*\|$	0.5973	0.0002
Observed behaviour	Zig-zags across the narrow valley	Damps the zig-zag and glides along the valley

Table 5 - Effect of the Momentum Coefficient β

S.No	Momentum β	Final Loss	Epochs to Loss $< 10^{-3}$	Oscillations	$\|w - w^*\|$	Remark
1	0.00	1.6934e-03	Not reached	50	0.5973	Plain gradient descent, does not converge
2	0.50	3.1536e-06	160	16	0.0258	Oscillation damped, tolerance reached
3	0.70	1.8151e-10	91	30	0.0002	Best balance of damping and speed
4	0.90	2.0195e-12	83	145	0.0000	Fastest loss decay but rings in w_1
5	0.95	2.3025e-06	124	143	0.0039	Heavy ringing, convergence slows
6	0.99	3.7803e+00	Not reached	141	2.1313	Unstable, diverges from the optimum

Plot 1 - Loss Curves of Standard Gradient Descent and Momentum



Plot 2 - Optimisation Path of both Methods on the Loss Surface

